

PRODUCTION HANDBOOK

Boforg Production Deployment Guide

Multi-Subdomain Django Deployment

Generated 8 August 2026

Ubuntu 24.04 LTS · Hostinger VPS · 72.60.20.46

Table of Contents

1. Current Architecture	4
2. Target Architecture	4
3. Pre-deployment Checklist	4
3.1 Open one persistent root session	5
3.2 Discover the existing deployment	5
3.3 Verify a clean production checkout	5
3.4 Create a dated rollback bundle	6
3.5 Back up PostgreSQL	6
4. DNS Configuration	7
4.1 Create these records in Cloudflare	7
4.2 Verify public DNS propagation	7
5. Server Configuration	7
5.1 Confirm required Ubuntu packages	7
5.2 Pull the intended existing project release	8
5.3 Install the release dependencies	8
5.4 Validate and migrate Django	8
5.5 Collect static files	8
6. Nginx Configuration	8
6.1 Discover and record the current Gunicorn upstream	9
6.2 Create the reusable proxy snippet	9
6.3 Create the complete subdomain site	10
6.4 Enable and validate the site	11
6.5 Verify origin routing before SSL	11
7. Gunicorn Configuration	11
7.1 Confirm the existing unit is correct	11
8. Django Configuration	12
8.1 Add only the required environment values	12
8.2 Run deployment checks with the production settings	12
9. SSL	13
9.1 Confirm DNS and ports before Certbot	13
9.2 Expand the existing certificate	13
9.3 Verify certificate and renewal	13

10. Testing	14
10.1 Verify DNS	14
10.2 Verify HTTPS and application routing	14
10.3 Bypass Cloudflare and verify the origin directly	14
10.4 Verify Django's shop health endpoint	14
10.5 Verify Gunicorn and Nginx	15
10.6 Verify static and media handling	15
10.7 Review logs	15
11. Troubleshooting	15
11.1 DNS returns no address or an old address	15
11.2 Nginx returns 502 Bad Gateway	15
11.3 Django returns 400 Bad Request	16
11.4 Login or forms return CSRF 403	16
11.5 Redirect loop after enabling Cloudflare	16
11.6 Static files return 404	16
11.7 Media files return 403 or 404	16
11.8 AI assistant fails while IMS works	17
12. Rollback	17
12.1 Load and validate the saved rollback path	17
12.2 Restore Nginx and its previous certificate references	17
12.3 Restore the Gunicorn environment drop-in	17
12.4 Restore the previous application commit	18
12.5 Restore PostgreSQL only for a full data rollback	18
12.6 Verify rollback	18
13. Final Verification Checklist	18

1. Current Architecture

The inspected repository is one Django 5.2 project with `manage.py` at its root, `config.settings` as its settings module, `config.urls` as its root URL configuration, and `config.wsgi.application` as the Gunicorn WSGI entry point. It uses one PostgreSQL database.

- Company website: `website`, mounted at `/`.
- Online shop: `shop`, mounted at `/shop/`; health endpoint: `/shop/healthz/`.
- IMS: `ims`, mounted at `/ims/`, with inventory, customers, sales, accounting, credit control, legal, and user dashboard modules.
- AI assistant: `tasker`, mounted inside the IMS at `/ims/tasker/`.
- Future API: no root API URL is currently defined; `api.boforg.co.zw` is reserved and deliberately returns HTTP 404 until an API URL is added.
- Static files: source directories `static/` and app static folders; collection target `staticfiles/`.
- Media files: `media/` under the project root.
- Production process: the repository README identifies the existing systemd service as `gunicorn-ims`.

The repository contains no checked-in Nginx site file or systemd unit. Therefore this handbook discovers the live project directory, service user, and Nginx upstream from the running VPS. It does not assume `/var/www`, `/srv`, a Unix-socket name, or a TCP port.

WARNING At inspection time, the local working tree contains uncommitted `AI/tasker` and related changes. Commit and push the intended release before running the VPS `git pull`. Do not deploy a different commit and expect `ai.boforg.co.zw` to work.

2. Target Architecture

```
Internet
  |
  v
Cloudflare DNS
  |
  v
72.60.20.46
  |
  v
Nginx (TLS termination and host routing)
  |
  v
Existing Gunicorn service: gunicorn-ims
  |
  v
One Boforg Django project / one PostgreSQL database
  |
  +-- boforg.co.zw      --> Company website at /
  +-- shop.boforg.co.zw --> redirect / to existing /shop/
  +-- ims.boforg.co.zw  --> redirect / to existing /ims/
  +-- ai.boforg.co.zw   --> redirect / to existing /ims/tasker/
  +-- api.boforg.co.zw  --> reserved; HTTP 404 JSON until implemented
```

DECISION The existing URL hierarchy is retained. Nginx gives each subdomain the correct landing page, while generated Django links continue to use the tested `/shop/`, `/ims/`, and `/ims/tasker/` paths. No duplicate project, app, database, or Gunicorn instance is created.

3. Pre-deployment Checklist

Complete every item before changing production.

- [] The intended local release is committed and pushed; `git status --short` is empty on the release branch.

- [] SSH/root access to [72.60.20.46](#) works.
- [] Cloudflare DNS access for [boforg.co.zw](#) works.
- [] PostgreSQL backup has been created and its file is non-empty.
- [] Nginx configuration has been backed up.
- [] [/etc/letsencrypt](#) has been backed up.
- [] Current Git commit and current Django migration state have been recorded.
- [] Existing website is reachable before the maintenance starts.

3.1 Open one persistent root session

COMMAND

```
ssh root@72.60.20.46
sudo -i
set -euo pipefail
```

PURPOSE

Opens the production VPS and enables fail-fast shell behavior. Keep this session open through Sections 3–10 because later commands reuse exported variables.

EXPECTED RESULT

The prompt is a root shell on the VPS and no command reports an error.

3.2 Discover the existing deployment

COMMAND

```
export BOFORG_SERVICE=gunicorn-ims
systemctl is-active --quiet "$BOFORG_SERVICE"
export BOFORG_PID="$(systemctl show -p MainPID --value "$BOFORG_SERVICE")"
export BOFORG_APP_DIR="$(systemctl show -p WorkingDirectory --value "$BOFORG_SERVICE")"
if [ -z "$BOFORG_APP_DIR" ]; then
    export BOFORG_APP_DIR="$(readlink -f "/proc/$BOFORG_PID/cwd")"
fi
export BOFORG_USER="$(systemctl show -p User --value "$BOFORG_SERVICE")"
export BOFORG_USER="${BOFORG_USER:-root}"
for candidate in "$BOFORG_APP_DIR/venv/bin/python" "$BOFORG_APP_DIR/.venv/bin/python"; do
    if [ -x "$candidate" ]; then export BOFORG_PYTHON="$candidate"; break; fi
done
test -f "$BOFORG_APP_DIR/manage.py"
test -x "$BOFORG_PYTHON"
mapfile -d '' -t BOFORG_ENV < "/proc/$BOFORG_PID/environ"
printf 'Service: %s\nPID: %s\nUser: %s\nProject: %s\nPython: %s\n' \
    "$BOFORG_SERVICE" "$BOFORG_PID" "$BOFORG_USER" "$BOFORG_APP_DIR" "$BOFORG_PYTHON"
```

PURPOSE

Obtains the real service, project, and virtual-environment paths from the live process.

EXPECTED RESULT

The printed project contains [manage.py](#); Python is inside its existing [venv](#) or [.venv](#).

3.3 Verify a clean production checkout

COMMAND

```
git -C "$BOFORG_APP_DIR" status --short --branch
test -z "$(git -C "$BOFORG_APP_DIR" status --porcelain)"
```

PURPOSE

Prevents [git pull](#) from overwriting or mixing uncommitted production changes.

EXPECTED RESULT

The branch is shown and the second command exits silently. Stop if it does not.

3.4 Create a dated rollback bundle

COMMAND

```
export BOFORG_BACKUP="/root/boforg-deploy-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$BOFORG_BACKUP"
printf '%s\n' "$BOFORG_BACKUP" > /root/boforg-deploy-last.path
git -C "$BOFORG_APP_DIR" rev-parse HEAD > "$BOFORG_BACKUP/git-head.txt"
git -C "$BOFORG_APP_DIR" status --short --branch > "$BOFORG_BACKUP/git-status.txt"
systemctl cat "$BOFORG_SERVICE" > "$BOFORG_BACKUP/gunicorn-ims.service.txt"
nginx -T > "$BOFORG_BACKUP/nginx-full.txt" 2>&1
tar -C / -czf "$BOFORG_BACKUP/nginx.tgz" etc/nginx
tar -C / -czf "$BOFORG_BACKUP/letsencrypt.tgz" etc/letsencrypt
```

PURPOSE

Records the release and creates restorable Nginx and SSL backups before any edit.

EXPECTED RESULT

`ls -lh "$BOFORG_BACKUP"` shows non-empty Git, service, Nginx, and LetsEncrypt files.

3.5 Back up PostgreSQL

COMMAND

```
cd "$BOFORG_APP_DIR"
export BOFORG_DB_NAME="$(sudo -u "$BOFORG_USER" env -i "${BOFORG_ENV[@]}" \
    "$BOFORG_PYTHON" manage.py shell -c \
    "from django.conf import settings; print(settings.DATABASES['default']['NAME'])")"
test -n "$BOFORG_DB_NAME"
printf '%s\n' "$BOFORG_DB_NAME" > "$BOFORG_BACKUP/db-name.txt"
sudo -u postgres pg_dump --format=custom --file="$BOFORG_BACKUP/postgresql.dump" "$BOFORG_DB_NAME"
test -s "$BOFORG_BACKUP/postgresql.dump"
sudo -u postgres pg_restore --list "$BOFORG_BACKUP/postgresql.dump" | head
```

PURPOSE

Reads the database name from the deployed Django settings and creates a PostgreSQL custom-format backup.

EXPECTED RESULT

The dump is non-empty and `pg_restore --list` prints archive entries.

4. DNS Configuration

The authoritative nameservers are currently Cloudflare (cory.ns.cloudflare.com and perla.ns.cloudflare.com). The apex and [www](#) already resolve through Cloudflare; the four requested subdomains did not resolve when this handbook was generated.

4.1 Create these records in Cloudflare

Type	Name	Content	Proxy status	TTL
A	shop	72.60.20.46	DNS only initially	Auto
A	ims	72.60.20.46	DNS only initially	Auto
A	ai	72.60.20.46	DNS only initially	Auto
A	api	72.60.20.46	DNS only initially	Auto

- [shop](#) sends shop traffic to the current Hostinger VPS.
- [ims](#) sends IMS traffic to the same VPS and Django project.
- [ai](#) sends AI-assistant traffic to the same VPS and Django project.
- [api](#) reserves the future API hostname on the same VPS.
- Keep the existing proxied apex [A](#) record and existing [www](#) record unchanged.

NOTE Use "DNS only" during certificate issuance and direct testing. After Section 10 passes, Cloudflare proxying may be enabled for each new record. Use SSL/TLS mode "Full (strict)".

WARNING Do not create an [AAAA](#) record unless this VPS has a configured public IPv6 address and Nginx is reachable over it. A wrong [AAAA](#) record causes intermittent failures.

4.2 Verify public DNS propagation

COMMAND

```
for host in shop.boforg.co.zw ims.boforg.co.zw ai.boforg.co.zw api.boforg.co.zw; do
    printf '%-24s ' "$host"
    dig +short A "$host" @1.1.1.1
done
```

PURPOSE

Checks the records through a public recursive resolver.

EXPECTED RESULT

Each hostname prints [72.60.20.46](#) while the Cloudflare proxy is disabled. If proxying is later enabled, Cloudflare anycast addresses are expected instead.

5. Server Configuration

5.1 Confirm required Ubuntu packages

COMMAND

```
apt-get update
apt-get install -y nginx postgresql-client certbot python3-certbot-nginx curl dnsutils
```

PURPOSE

Ensures the existing stack has the command-line tools used by this handbook. Installed packages are retained and upgraded only according to Ubuntu package policy.

EXPECTED RESULT

APT completes without unresolved dependencies.

5.2 Pull the intended existing project release

COMMAND

```
git -C "$BOFORG_APP_DIR" fetch --prune origin
git -C "$BOFORG_APP_DIR" pull --ff-only
git -C "$BOFORG_APP_DIR" status --short --branch
```

PURPOSE

Updates the existing checkout without creating a project or allowing a merge commit on production.

EXPECTED RESULT

Git reports a fast-forward or “Already up to date”, followed by a clean branch.

5.3 Install the release dependencies

COMMAND

```
sudo -u "$BOFORG_USER" "$BOFORG_PYTHON" -m pip install --requirement "$BOFORG_APP_DIR/requirements.txt"
```

PURPOSE

Updates the existing virtual environment, including Gunicorn, OpenAI, PostgreSQL, and PDF dependencies already declared by the project.

EXPECTED RESULT

Pip completes without an error. No new virtual environment is created.

5.4 Validate and migrate Django

COMMAND

```
cd "$BOFORG_APP_DIR"
sudo -u "$BOFORG_USER" env -i "${BOFORG_ENV[@]}" "$BOFORG_PYTHON" manage.py check
sudo -u "$BOFORG_USER" env -i "${BOFORG_ENV[@]}" "$BOFORG_PYTHON" manage.py migrate --plan
sudo -u "$BOFORG_USER" env -i "${BOFORG_ENV[@]}" "$BOFORG_PYTHON" manage.py migrate --noinput
```

PURPOSE

Checks the integrated project and applies only its pending migrations to the existing PostgreSQL database.

EXPECTED RESULT

The check reports no issues; migrations either apply successfully or report that none are pending.

5.5 Collect static files

COMMAND

```
cd "$BOFORG_APP_DIR"
sudo -u "$BOFORG_USER" env -i "${BOFORG_ENV[@]}" \
  "$BOFORG_PYTHON" manage.py collectstatic --noinput
test -d "$BOFORG_APP_DIR/staticfiles"
find "$BOFORG_APP_DIR/staticfiles" -type f | head
```

PURPOSE

Builds the production `staticfiles/` directory configured in `config/settings.py`.

EXPECTED RESULT

Collectstatic completes and the final command lists collected files.

6. Nginx Configuration

The existing apex-domain Nginx site remains in place. The following files add only the four new subdomains and reuse the live site’s actual `proxy_pass` target.

6.1 Discover and record the current Gunicorn upstream

COMMAND

```
export BOFORG_PROXY_PASS="$(nginx -T 2>/dev/null | awk '
  /server_name/ && /boforg\.co\.zw/ { found=1 }
  found && $1=="proxy_pass" { gsub(/;/, "", $2); print $2; exit }
  ')"
test -n "$BOFORG_PROXY_PASS"
printf 'Reusing proxy_pass %s\n' "$BOFORG_PROXY_PASS"
```

PURPOSE

Extracts the actual socket, local port, or named upstream already serving boforg.co.zw.

EXPECTED RESULT

A single value such as <http://unix:/run/gunicorn-ims.sock> or <http://127.0.0.1:8000> is printed. Stop if it is empty.

6.2 Create the reusable proxy snippet

FILE

</etc/nginx/snippets/boforg-proxy.conf>

COMMAND

```
cat > /etc/nginx/snippets/boforg-proxy.conf <<'NGINX'
proxy_pass __BOFORG_PROXY_PASS__;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
proxy_redirect off;
proxy_connect_timeout 30s;
proxy_read_timeout 120s;
NGINX
sed -i "s|__BOFORG_PROXY_PASS__|$BOFORG_PROXY_PASS|" /etc/nginx/snippets/boforg-proxy.conf
cat /etc/nginx/snippets/boforg-proxy.conf
```

PURPOSE

Creates one proxy definition using the existing Gunicorn upstream and standard forwarded headers required by Django.

EXPECTED RESULT

The file contains the discovered [proxy_pass](#); no placeholder remains.

6.3 Create the complete subdomain site

FILE

/etc/nginx/sites-available/boforg-subdomains

COMMAND

```
cat > /etc/nginx/sites-available/boforg-subdomains <<'NGINX'
server {
    listen 80;
    listen [::]:80;
    server_name shop.boforg.co.zw;

    access_log /var/log/nginx/boforg-shop.access.log;
    error_log /var/log/nginx/boforg-shop.error.log;
    client_max_body_size 25m;

    location = / { return 302 /shop/; }
    location /static/ { alias __BOFORG_APP_DIR__/staticfiles/; access_log off; expires 30d; }
    location /media/ { alias __BOFORG_APP_DIR__/media/; expires 7d; }
    location / { include /etc/nginx/snippets/boforg-proxy.conf; }
}

server {
    listen 80;
    listen [::]:80;
    server_name ims.boforg.co.zw;

    access_log /var/log/nginx/boforg-ims.access.log;
    error_log /var/log/nginx/boforg-ims.error.log;
    client_max_body_size 25m;

    location = / { return 302 /ims/; }
    location /static/ { alias __BOFORG_APP_DIR__/staticfiles/; access_log off; expires 30d; }
    location /media/ { alias __BOFORG_APP_DIR__/media/; expires 7d; }
    location / { include /etc/nginx/snippets/boforg-proxy.conf; }
}

server {
    listen 80;
    listen [::]:80;
    server_name ai.boforg.co.zw;

    access_log /var/log/nginx/boforg-ai.access.log;
    error_log /var/log/nginx/boforg-ai.error.log;
    client_max_body_size 25m;

    location = / { return 302 /ims/tasker/; }
    location /static/ { alias __BOFORG_APP_DIR__/staticfiles/; access_log off; expires 30d; }
    location /media/ { alias __BOFORG_APP_DIR__/media/; expires 7d; }
    location / { include /etc/nginx/snippets/boforg-proxy.conf; }
}

server {
    listen 80;
    listen [::]:80;
    server_name api.boforg.co.zw;

    access_log /var/log/nginx/boforg-api.access.log;
    error_log /var/log/nginx/boforg-api.error.log;
    default_type application/json;
    return 404 '{"detail":"Boforg API is reserved for future deployment."}';
}
NGINX
sed -i "s|__BOFORG_APP_DIR__|${BOFORG_APP_DIR}|g" /etc/nginx/sites-available/boforg-subdomains
test -z "$(grep '__BOFORG__' /etc/nginx/sites-available/boforg-subdomains)"
```

PURPOSE

Creates four hostname-specific HTTP server blocks without altering the working apex-domain site.

EXPECTED RESULT

The file contains the actual project path and no placeholder text.

6.4 Enable and validate the site

COMMAND

```
ln -sfn /etc/nginx/sites-available/boforg-subdomains \
    /etc/nginx/sites-enabled/boforg-subdomains
nginx -t
systemctl reload nginx
systemctl is-active --quiet nginx
```

PURPOSE

Enables the subdomain configuration only after a syntax test, then reloads Nginx without stopping active connections.

EXPECTED RESULT

Nginx reports `syntax is ok`, `test is successful`, and remains active.

6.5 Verify origin routing before SSL

COMMAND

```
curl -sSI -H 'Host: shop.boforg.co.zw' http://127.0.0.1/ | head
curl -sSI -H 'Host: ims.boforg.co.zw' http://127.0.0.1/ | head
curl -sSI -H 'Host: ai.boforg.co.zw' http://127.0.0.1/ | head
curl -si -H 'Host: api.boforg.co.zw' http://127.0.0.1/ | head
```

PURPOSE

Tests Nginx host matching locally, independent of DNS.

EXPECTED RESULT

Shop, IMS, and AI return HTTP 302 to `/shop/`, `/ims/`, and `/ims/tasker/`; API returns HTTP 404 JSON.

7. Unicorn Configuration

No Unicorn command, bind address, worker count, user, virtual environment, or service file must change. All host routing occurs in Nginx, and every hostname reaches the same `config.wsgi.application` and PostgreSQL database.

7.1 Confirm the existing unit is correct

COMMAND

```
systemctl cat "$BOFORG_SERVICE"
systemctl show "$BOFORG_SERVICE" -p User -p WorkingDirectory -p ExecStart
systemctl is-enabled "$BOFORG_SERVICE"
systemctl is-active "$BOFORG_SERVICE"
```

PURPOSE

Confirms the handbook is reusing the current service rather than replacing it.

EXPECTED RESULT

The unit is enabled and active; its working directory equals `$BOFORG_APP_DIR`, and its command ultimately loads `config.wsgi:application` or `config.wsgi.application`.

WARNING

If the WSGI target is not `config.wsgi`, stop. That would conflict with the inspected project and requires a separate review of the live unit.

8. Django Configuration

No Python source edit is required. `config/settings.py` already reads `DJANGO_ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`, `PUBLIC_BASE_URL`, and `SHOP_PUBLIC_BASE`-related integrations from environment variables, trusts `X-Forwarded-Proto`, and enables production cookie and HTTPS settings when `DJANGO_DEBUG=False`.

8.1 Add only the required environment values

FILE

`/etc/systemd/system/gunicorn-ims.service.d/subdomains.conf`

COMMAND

```
mkdir -p /etc/systemd/system/gunicorn-ims.service.d
if [ -f /etc/systemd/system/gunicorn-ims.service.d/subdomains.conf ]; then
  cp -a /etc/systemd/system/gunicorn-ims.service.d/subdomains.conf \
    "$BOFORG_BACKUP/subdomains.conf.before"
fi
cat > /etc/systemd/system/gunicorn-ims.service.d/subdomains.conf <<'SYSTEMD'
[Service]
Environment="DJANGO_ALLOWED_HOSTS=.boforg.co.zw,localhost,127.0.0.1"
Environment="CSRF_TRUSTED_ORIGINS=https://boforg.co.zw,https://*.boforg.co.zw"
Environment="PUBLIC_BASE_URL=https://boforg.co.zw"
Environment="SHOP_PUBLIC_BASE=https://shop.boforg.co.zw"
Environment="DJANGO_DEBUG=False"
Environment="SECURE_SSL_REDIRECT=True"
SYSTEMD
systemctl daemon-reload
systemctl restart "$BOFORG_SERVICE"
systemctl is-active --quiet "$BOFORG_SERVICE"
```

PURPOSE

Allows all five public hostnames and their HTTPS CSRF origins while retaining existing secrets and database configuration.

EXPECTED RESULT

The service restarts and remains active.

8.2 Run deployment checks with the production settings

COMMAND

```
cd "$BOFORG_APP_DIR"
sudo -u "$BOFORG_USER" env -i \
  "${BOFORG_ENV[@]}" \
  DJANGO_ALLOWED_HOSTS='.boforg.co.zw,localhost,127.0.0.1' \
  CSRF_TRUSTED_ORIGINS='https://boforg.co.zw,https://*.boforg.co.zw' \
  DJANGO_DEBUG=False "$BOFORG_PYTHON" manage.py check --deploy
```

PURPOSE

Runs Django's production security checks with the same host and origin values as Gunicorn.

EXPECTED RESULT

No errors appear. Review warnings; a warning about an intentionally external setting must be understood before continuing.

NOTE

Do not put `DJANGO_SECRET_KEY`, database passwords, Paynow credentials, Meta credentials, or the OpenAI key in this handbook or Git. Keep the existing secret source used by `gunicorn-ims`.

9. SSL

9.1 Confirm DNS and ports before Certbot

COMMAND

```
for host in \  
  boforg.co.zw www.boforg.co.zw shop.boforg.co.zw \  
  ims.boforg.co.zw ai.boforg.co.zw api.boforg.co.zw  
do  
  dig +short A "$host" @1.1.1.1  
done  
ufw status
```

PURPOSE

Confirms every certificate name resolves and that HTTP/HTTPS are not blocked.

EXPECTED RESULT

Every hostname resolves; firewall policy allows Nginx Full or TCP ports 80 and 443.

9.2 Expand the existing certificate

COMMAND

```
export BOFORG_CERT_NAME="$(certbot certificates 2>/dev/null | awk '  
  /Certificate Name:/ { name=$3 }  
  /Domains:/ && /boforg\.co\.zw/ { print name; exit }  
' )"  
test -n "$BOFORG_CERT_NAME"  
certbot --nginx --redirect --expand --cert-name "$BOFORG_CERT_NAME" \  
  -d boforg.co.zw \  
  -d www.boforg.co.zw \  
  -d shop.boforg.co.zw \  
  -d ims.boforg.co.zw \  
  -d ai.boforg.co.zw \  
  -d api.boforg.co.zw
```

PURPOSE

Expands the existing Boforg LetsEncrypt certificate and lets Certbot add HTTPS plus HTTP-to-HTTPS redirects to the relevant Nginx blocks.

EXPECTED RESULT

Certbot reports success and lists all six names on the certificate.

9.3 Verify certificate and renewal

COMMAND

```
nginx -t  
systemctl reload nginx  
certbot certificates  
certbot renew --dry-run  
systemctl status certbot.timer --no-pager
```

PURPOSE

Checks the post-Certbot Nginx configuration, certificate coverage, and unattended renewal.

EXPECTED RESULT

The certificate includes all names, dry-run renewal succeeds, and the timer is active.

NOTE

After all tests pass, turn Cloudflare proxying on for the four new records and set Cloudflare SSL/TLS encryption mode to "Full (strict)". Never use "Flexible", which causes HTTPS redirect loops.

10. Testing

10.1 Verify DNS

COMMAND

```
for host in boforg.co.zw shop.boforg.co.zw ims.boforg.co.zw ai.boforg.co.zw api.boforg.co.zw; do
    printf '\n%s\n' "$host"
    dig +short A "$host" @1.1.1.1
done
```

PURPOSE

Confirms public resolution after the final Cloudflare setting is applied.

EXPECTED RESULT

Each name returns the origin IP when DNS-only, or Cloudflare addresses when proxied.

10.2 Verify HTTPS and application routing

COMMAND

```
curl -fsSIL --max-redirs 5 https://boforg.co.zw/ | head -n 12
curl -fsSIL --max-redirs 5 https://shop.boforg.co.zw/ | head -n 12
curl -fsSIL --max-redirs 5 https://ims.boforg.co.zw/ | head -n 12
curl -fsSIL --max-redirs 5 https://ai.boforg.co.zw/ | head -n 12
curl -sS -o /dev/null -w 'API HTTP %{http_code}\n' https://api.boforg.co.zw/
```

PURPOSE

Tests the public certificate, redirects, and final application responses.

EXPECTED RESULT

Website, shop, IMS, and AI finish without a TLS or 5xx error. API prints **API HTTP 404** by design.

10.3 Bypass Cloudflare and verify the origin directly

COMMAND

```
for host in boforg.co.zw shop.boforg.co.zw ims.boforg.co.zw ai.boforg.co.zw api.boforg.co.zw; do
    curl --resolve "$host:443:72.60.20.46" -sS -o /dev/null \
        -w "$host origin HTTP %{http_code}\n" "https://$host/"
done
```

PURPOSE

Tests Nginx and LetsEncrypt directly so a Cloudflare response cannot hide an origin fault.

EXPECTED RESULT

The first four names return 200 or 302; API returns 404.

10.4 Verify Django's shop health endpoint

COMMAND

```
curl --resolve shop.boforg.co.zw:443:72.60.20.46 -fsS \
    https://shop.boforg.co.zw/shop/healthz/
printf '\n'
```

PURPOSE

Exercises Nginx, Gunicorn, Django URL routing, and the application health view in one request.

EXPECTED RESULT

The health endpoint returns successfully without a 4xx or 5xx response.

10.5 Verify Gunicorn and Nginx

COMMAND

```
systemctl is-enabled "$BOFORG_SERVICE"
systemctl is-active "$BOFORG_SERVICE"
systemctl is-enabled nginx
systemctl is-active nginx
ss -lntp | grep -E ':(80|443)\b'
```

PURPOSE

Confirms both services start at boot, are running now, and Nginx listens on HTTP and HTTPS.

EXPECTED RESULT

Both services are enabled and active; ports 80 and 443 are listening.

10.6 Verify static and media handling

COMMAND

```
test -s "$BOFORG_APP_DIR/staticfiles/css/app.css"
curl --resolve boforg.co.zw:443:72.60.20.46 -fsSI \
  https://boforg.co.zw/static/css/app.css | head
find "$BOFORG_APP_DIR/media" -type f -print -quit 2>/dev/null || true
```

PURPOSE

Checks a known static asset and confirms whether production media content exists.

EXPECTED RESULT

The static request returns HTTP 200. If media files exist, verify one of their exact [/media/...](#) URLs in a browser.

10.7 Review logs

COMMAND

```
journalctl -u "$BOFORG_SERVICE" --since '15 minutes ago' --no-pager
tail -n 100 /var/log/nginx/boforg-shop.error.log
tail -n 100 /var/log/nginx/boforg-ims.error.log
tail -n 100 /var/log/nginx/boforg-ai.error.log
tail -n 100 /var/log/nginx/error.log
```

PURPOSE

Checks the deployment window for Django tracebacks, upstream failures, permission errors, and Nginx configuration problems.

EXPECTED RESULT

No new traceback, repeated 5xx, upstream-connect, or permission-denied error appears.

11. Troubleshooting

11.1 DNS returns no address or an old address

Diagnosis

```
dig +trace shop.boforg.co.zw
dig +short A shop.boforg.co.zw @1.1.1.1
```

Fix

Correct the Cloudflare [A](#) record, remove conflicting records, leave it DNS-only, and wait for resolver caches. Do not continue to Certbot until public DNS reaches [72.60.20.46](#).

11.2 Nginx returns 502 Bad Gateway

Diagnosis

```
systemctl status "$BOFORG_SERVICE" --no-pager
journalctl -u "$BOFORG_SERVICE" -n 100 --no-pager
grep proxy_pass /etc/nginx/snippets/boforg-proxy.conf
nginx -T 2>/dev/null | grep -n 'proxy_pass'
```

Fix

Start or repair `gunicorn-ims`. If its bind target changed, replace only the `proxy_pass` value in `/etc/nginx/snippets/boforg-proxy.conf`, run `nginx -t`, and reload Nginx.

11.3 Django returns 400 Bad Request

Diagnosis

```
systemctl show "$BOFORG_SERVICE" -p Environment | tr ' ' '\n' | grep DJANGO_ALLOWED_HOSTS
journalctl -u "$BOFORG_SERVICE" -n 100 --no-pager | grep -i DisallowedHost
```

Fix

Correct `DJANGO_ALLOWED_HOSTS` in the systemd drop-in, then run `systemctl daemon-reload` and restart `gunicorn-ims`.

11.4 Login or forms return CSRF 403

Diagnosis

```
systemctl show "$BOFORG_SERVICE" -p Environment | tr ' ' '\n' | grep CSRF_TRUSTED_ORIGINS
journalctl -u "$BOFORG_SERVICE" -n 100 --no-pager | grep -i csrf
```

Fix

Ensure the exact `https://` origin is present, Nginx sends `X-Forwarded-Proto $scheme`, then restart Gunicorn. Clear stale browser cookies and retry.

11.5 Redirect loop after enabling Cloudflare

Diagnosis

```
curl -IL --max-redirs 10 https://shop.boforg.co.zw/
```

Fix

Set Cloudflare SSL/TLS mode to `Full (strict)`, not `Flexible`. Keep Django's `SECURE_PROXY_SSL_HEADER` and Nginx's `X-Forwarded-Proto` header unchanged.

11.6 Static files return 404

Diagnosis

```
ls -l "$BOFORG_APP_DIR/staticfiles/css/app.css"
namei -l "$BOFORG_APP_DIR/staticfiles/css/app.css"
nginx -T 2>/dev/null | grep -A2 'location /static/'
```

Fix

Run `collectstatic` again. Ensure every parent directory is traversable by the Nginx worker and the alias ends with `/`; then test and reload Nginx.

11.7 Media files return 403 or 404

Diagnosis

```
namei -l "$BOFORG_APP_DIR/media"
tail -n 100 /var/log/nginx/boforg-ims.error.log
```

Fix

Confirm the file exists under the configured `MEDIA_ROOT`. Grant the Nginx worker read/traverse access using the server's existing ownership or ACL policy; do not make uploaded files world-writable.

11.8 AI assistant fails while IMS works

Diagnosis

```
journalctl -u "$BOFORG_SERVICE" -n 200 --no-pager | grep -Ei 'tasker|openai|traceback'
cd "$BOFORG_APP_DIR"
sudo -u "$BOFORG_USER" "$BOFORG_PYTHON" manage.py showmigrations tasker
```

Fix

Confirm the release contains the `tasker` package and migrations, apply pending migrations, and verify the existing secret source supplies `OPENAI_API_KEY`. Never paste the key into Nginx or Git.

12. Rollback

Use rollback if a required production check fails and cannot be corrected immediately.

12.1 Load and validate the saved rollback path

COMMAND

```
set -euo pipefail
export BOFORG_BACKUP="$(cat /root/boforg-deploy-last.path)"
test -d "$BOFORG_BACKUP"
test -s "$BOFORG_BACKUP/nginx.tgz"
test -s "$BOFORG_BACKUP/postgresql.dump"
cat "$BOFORG_BACKUP/git-head.txt"
```

PURPOSE

Ensures rollback uses the exact backup created immediately before this deployment.

EXPECTED RESULT

The backup is valid and the previous Git commit is printed.

12.2 Restore Nginx and its previous certificate references

COMMAND

```
rm -f /etc/nginx/sites-enabled/boforg-subdomains
rm -f /etc/nginx/sites-available/boforg-subdomains
rm -f /etc/nginx/snippets/boforg-proxy.conf
tar -C / -xzf "$BOFORG_BACKUP/nginx.tgz"
nginx -t
systemctl reload nginx
```

PURPOSE

Removes only the files created by this handbook and restores the complete pre-change Nginx tree. The expanded certificate files remain harmless and may be reused later.

EXPECTED RESULT

The original apex site passes `nginx -t` and reloads.

12.3 Restore the Gunicorn environment drop-in

COMMAND

```
if [ -f "$BOFORG_BACKUP/subdomains.conf.before" ]; then
  cp -a "$BOFORG_BACKUP/subdomains.conf.before" \
    /etc/systemd/system/gunicorn-ims.service.d/subdomains.conf
else
  rm -f /etc/systemd/system/gunicorn-ims.service.d/subdomains.conf
fi
systemctl daemon-reload
systemctl restart gunicorn-ims
systemctl is-active --quiet gunicorn-ims
```

PURPOSE

Returns Gunicorn's hostname environment to its exact prior state.

EXPECTED RESULT

Gunicorn restarts and is active.

12.4 Restore the previous application commit

COMMAND

```
export BOFORG_APP_DIR="$(systemctl show -p WorkingDirectory --value gunicorn-ims)"
if [ -z "$BOFORG_APP_DIR" ]; then
    export BOFORG_PID="$(systemctl show -p MainPID --value gunicorn-ims)"
    export BOFORG_APP_DIR="$(readlink -f "/proc/$BOFORG_PID/cwd")"
fi
git -C "$BOFORG_APP_DIR" status --short
git -C "$BOFORG_APP_DIR" switch --detach "$(cat "$BOFORG_BACKUP/git-head.txt")"
systemctl restart gunicorn-ims
```

PURPOSE

Checks out the exact pre-deployment release without deleting the branch or creating another project.

EXPECTED RESULT

Git reports detached HEAD at the recorded commit and Gunicorn restarts.

WARNING Stop if `git status --short` is not empty. Preserve and review unexpected production edits before changing the checkout.

12.5 Restore PostgreSQL only for a full data rollback

WARNING This operation deletes all database changes made after the backup. Stop Gunicorn, notify users, and use it only when application rollback alone is insufficient.

COMMAND

```
export BOFORG_DB_NAME="$(cat "$BOFORG_BACKUP/db-name.txt")"
test -n "$BOFORG_DB_NAME"
systemctl stop gunicorn-ims
sudo -u postgres dropdb "$BOFORG_DB_NAME"
sudo -u postgres createdb "$BOFORG_DB_NAME"
sudo -u postgres pg_restore --exit-on-error --dbname="$BOFORG_DB_NAME" \
    "$BOFORG_BACKUP/postgresql.dump"
systemctl start gunicorn-ims
systemctl is-active --quiet gunicorn-ims
```

PURPOSE

Replaces the database with the exact pre-deployment snapshot when migrations or data changes require full reversal.

EXPECTED RESULT

Restore finishes without error and Gunicorn becomes active.

12.6 Verify rollback

COMMAND

```
nginx -t
systemctl is-active nginx
systemctl is-active gunicorn-ims
curl -fsSIL --max-redirs 5 https://boforg.co.zw/ | head
```

PURPOSE

Confirms the original public site is restored.

EXPECTED RESULT

Both services are active and the apex site responds without a 5xx error.

13. Final Verification Checklist

[] <https://boforg.co.zw/> displays the company website.

- [] <https://shop.boforg.co.zw/> reaches the existing online shop.
- [] <https://ims.boforg.co.zw/> reaches the existing IMS/login flow.
- [] <https://ai.boforg.co.zw/> reaches the existing Tasker AI assistant/login flow.
- [] <https://api.boforg.co.zw/> returns the intentional HTTP 404 JSON reservation response.
- [] All public hostnames use a valid certificate with no browser warning.
- [] Cloudflare SSL/TLS mode is `Full (strict)` if proxying is enabled.
- [] Nginx is enabled, active, and passes `nginx -t`.
- [] `gunicorn-ims` is enabled and active.
- [] Django `/shop/healthz/` succeeds through HTTPS.
- [] Static files return HTTP 200.
- [] An existing media file loads over HTTPS, if production media exists.
- [] PostgreSQL migrations are current.
- [] Gunicorn journal contains no new traceback.
- [] Nginx error logs contain no repeated 4xx/5xx, upstream, or permission errors.
- [] The dated rollback bundle remains stored securely until the release is accepted.

NOTE

Mail-server installation is intentionally excluded. This handbook does not install or configure Mailcow or any other mail service.